

FinScript – Projeto e Implementação de uma Linguagem de Domínio Específico para Processamento de Dados Financeiros

Felipe Gustavo Pinheiro
Fundação Hermínio Ometto
Araras, Brasil

Feliipepinheiro09@alunos.fho.edu.br

João Pedro Rebelato
Fundação Hermínio Ometto
Araras, Brasil

Joaopedrorebe7756@alunos.fho.edu.br

Julio Cezar Da Cunha
Fundação Hermínio Ometto
Araras, Brasil

Juliocezar@alunos.fho.edu.br

Abstract—Este artigo apresenta o projeto e a implementação da FinScript, uma Linguagem de Domínio Específico (DSL) voltada ao processamento de dados financeiros. Diante da dificuldade que analistas financeiros enfrentam ao utilizar linguagens de propósito geral para expressar operações como declaração de ativos, simulação de portfólios e geração de relatórios de rendimento, propõe-se uma linguagem com sintaxe declarativa próxima ao vocabulário do domínio. O trabalho contempla as etapas clássicas de um compilador: definição da gramática formal em EBNF, análise léxica (*scanner*), análise sintática (*parser* descendente recursivo), análise semântica com verificação de tipos e tabela de símbolos, e execução via caminhamento na Árvore Sintática Abstrata (AST). A implementação foi realizada em Python e a validação ocorreu por meio de casos de uso representativos do mercado financeiro brasileiro, incluindo simulações de renda fixa indexadas ao CDI e SELIC, portfólios de ações e fundos, e estruturas condicionais para parametrização de simulações. Os resultados demonstram que a FinScript reduz substancialmente a verbosidade necessária para expressar operações financeiras em comparação a linguagens de propósito geral, ao mesmo tempo em que incorpora regras de negócio do mercado brasileiro — como a tabela regressiva de IR e os indexadores CDI, IPCA e SELIC — diretamente na semântica da linguagem.

Index Terms—linguagem de domínio específico, compiladores, gramática formal, EBNF, processamento financeiro, análise léxica, análise sintática, Python

I. INTRODUÇÃO

O setor financeiro é um dos domínios mais exigentes da engenharia de software no que diz respeito à precisão semântica, à rastreabilidade de regras de negócio e à expressividade das linguagens utilizadas. Operações como cálculo de juros compostos, simulação de portfólios, análise de fluxo de caixa e geração de relatórios gerenciais exigem que o código que as descreve seja não apenas correto, mas também legível e auditável por profissionais que nem sempre possuem fluência em linguagens de programação de propósito geral, como Python ou Java.

Nesse contexto, as Linguagens de Domínio Específico — *Domain-Specific Languages* (DSLs) — emergem como uma solução consolidada na engenharia de software. Segundo [1],

uma DSL é uma linguagem de programação de expressividade limitada, focada em um domínio particular, que oferece ganhos substanciais em expressividade e facilidade de uso em comparação a linguagens de propósito geral dentro de sua área de aplicação. Ao restringir o escopo sintático e semântico ao domínio financeiro, é possível criar uma linguagem cujos construtores reflitam diretamente os conceitos operados por analistas, gestores e engenheiros financeiros no dia a dia.

A ideia de aplicar técnicas de linguagens de programação ao domínio financeiro não é recente. [2] demonstraram que conceitos da teoria de linguagens se aplicam diretamente ao problema da descrição e avaliação de contratos financeiros, propondo uma biblioteca de combinadores capaz de descrever instrumentos financeiros com semântica denotacional rigorosa. Esse trabalho seminal evidenciou o potencial das DSLs no setor e abriu caminho para soluções comerciais e acadêmicas subsequentes.

Apesar dos avanços, a maioria das ferramentas disponíveis para análise financeira programática ainda exige que o usuário opere em linguagens de propósito geral, impondo uma curva de aprendizado significativa e afastando especialistas do domínio do processo de definição das regras de negócio. Ferramentas como planilhas eletrônicas são acessíveis, mas carecem de expressividade formal, controle de versão e capacidade de verificação semântica.

Este trabalho propõe o projeto e a implementação da **FinScript**, uma DSL externa voltada ao processamento de dados financeiros. A linguagem permitirá declarar ativos de diferentes classes — renda fixa, ações e fundos —, compor portfólios, simular rendimentos ao longo do tempo, aplicar verificações semânticas de domínio e gerar relatórios estruturados. O interpretador será desenvolvido em Python, contemplando as fases clássicas de compilação: análise léxica, análise sintática por *parser* descendente recursivo, análise semântica com tabela de símbolos e execução via caminhamento na AST.

II. OBJETIVOS

O objetivo geral deste trabalho é projetar e implementar a *FinScript*, uma Linguagem de Domínio Específico para processamento de dados financeiros, acompanhada de um interpretador funcional capaz de executar programas escritos nessa linguagem, realizando verificações semânticas de domínio e produzindo relatórios de rendimento como saída.

Os objetivos específicos que norteiam o desenvolvimento são:

- Definir a gramática formal da *FinScript* em notação EBNF, abrangendo construções para declaração de ativos, composição de portfólios, definição de parâmetros de simulação e geração de relatórios;
- Implementar a fase de análise léxica por meio de um *scanner* em Python, responsável por identificar e classificar os *tokens* da linguagem;
- Implementar a fase de análise sintática por meio de um *parser* descendente recursivo, capaz de validar a estrutura gramatical dos programas e construir a AST correspondente;
- Implementar a fase de análise semântica com tabela de símbolos, verificação de tipos e regras de negócio do domínio financeiro, como validação de taxas, datas de vencimento e concentração de risco em portfólios;
- Desenvolver o motor de interpretação via caminhamento na AST, produzindo relatórios de rendimento e alertas semânticos como saída;
- Validar a linguagem por meio de casos de uso representativos do mercado financeiro brasileiro, demonstrando sua adequação ao domínio em comparação a soluções em linguagem de propósito geral.

III. JUSTIFICATIVA

A criação de uma DSL para processamento de dados financeiros se justifica por razões técnicas, operacionais e acadêmicas.

Do ponto de vista técnico, linguagens de propósito geral como Python ou Java, embora poderosas, impõem ao desenvolvedor a necessidade de implementar toda a infraestrutura de validação, tipagem e semântica do domínio financeiro a partir do zero. Segundo [1], DSLs bem projetadas preenchem a lacuna entre programadores e especialistas de domínio ao fornecer notações e construtores familiares, tornando os programas resultantes compreensíveis e modificáveis pelo próprio especialista.

Do ponto de vista operacional, o mercado financeiro brasileiro apresenta particularidades que tornam ainda mais relevante a existência de uma linguagem dedicada. Produtos como CDB, LCI, LCA, Tesouro Direto e portfólios de ações na B3 possuem regras de remuneração específicas — como indexação ao CDI, ao IPCA ou a taxas pré-fixadas — que exigem lógica de cálculo precisa e estruturas de dados adequadas. Além disso, [3] descreve como a adoção de uma DSL em ambiente corporativo financeiro resultou em ajustes simplificados de produtos e melhoria na precisão de relatórios, evidenciando os benefícios práticos dessa abordagem.

Do ponto de vista acadêmico, o desenvolvimento da *FinScript* representa uma oportunidade concreta de aplicar, de forma integrada, os conceitos fundamentais de Teoria da Computação e Compiladores: autômatos finitos na especificação do léxico, gramáticas livres de contexto na definição da sintaxe, tabelas de símbolos e verificação de tipos na análise semântica, e caminhamento em árvore na fase de interpretação.

IV. REVISÃO BIBLIOGRÁFICA

Esta seção apresenta os trabalhos relacionados que fundamentam o desenvolvimento da *FinScript*, organizados em três eixos: (i) teoria e metodologia de desenvolvimento de DSLs; (ii) aplicações de DSLs no domínio financeiro; e (iii) DSLs para processamento intensivo de dados de domínio específico.

A. Teoria e Metodologia de Desenvolvimento de DSLs

A referência consolidada sobre quando e como desenvolver DSLs é o trabalho de [1]. Os autores apresentam um conjunto de padrões de decisão, análise, projeto e implementação de DSLs, argumentando que o desenvolvimento de uma DSL é justificado quando a linguagem permite que um tipo específico de problema seja expresso com mais clareza do que seria possível com uma linguagem de propósito geral, e quando esse tipo de problema reaparece com frequência suficiente para amortizar o custo de desenvolvimento da linguagem. No caso do processamento financeiro, ambas as condições são satisfeitas: a expressividade de conceitos como portfólios, taxas e vencimentos é limitada em linguagens gerais, e as operações financeiras são recorrentes em qualquer organização que gerencie recursos.

Os autores também classificam as DSLs em externas — com sintaxe própria, processadas por compilador ou interpretador independente — e internas — implementadas como bibliotecas em uma linguagem hospedeira. A *FinScript* se enquadra como DSL externa, pois possui sintaxe e semântica próprias, processadas por um interpretador dedicado.

B. DSLs Aplicadas ao Domínio Financeiro

O trabalho seminal de [2] é o ponto de partida obrigatório para qualquer discussão sobre DSLs financeiras. Os autores propuseram uma biblioteca de combinadores em Haskell capaz de descrever contratos financeiros — incluindo opções, swaps, futuros e instrumentos compostos — com semântica denotacional rigorosa. O principal resultado do trabalho é que um conjunto pequeno de combinadores primitivos, quando composto, é capaz de descrever a vasta maioria dos instrumentos financeiros encontrados na prática. Esse achado demonstra que o domínio financeiro possui estrutura combinatória natural que se presta bem à definição de uma linguagem formal.

No contexto industrial, [3] descreve a implementação da *Finance Modeling Language* (FML), uma DSL desenvolvida para modelar produtos financeiros em ambiente corporativo. O trabalho relata que a adoção da FML resultou em maior facilidade para ajustes e cálculos de cenários, além de melhoria na precisão dos relatórios financeiros. Essa experiência industrial valida as escolhas de escopo e design adotadas na *FinScript*.

C. DSLs para Processamento Intensivo de Dados de Domínio

[4] apresentam a SWEL (*Scientific Workflow Execution Language*), uma DSL de modelagem para especificação de fluxos de trabalho intensivos em dados, desenvolvida segundo princípios de engenharia orientada a modelos. O trabalho é relevante para a *FinScript* por duas razões principais. Primeiro, a metodologia adotada pelos autores — identificação dos elementos centrais do domínio junto a especialistas, definição de gramática formal independente de plataforma e validação por casos de uso reais — é diretamente aplicável ao processo de desenvolvimento da *FinScript*. Segundo, o trabalho demonstra que DSLs para processamento de dados específicos de domínio representam uma linha de pesquisa ativa e consolidada na literatura de engenharia de software.

Complementarmente, [5] investigam a construção de modelos de linguagem específicos para mineração de texto financeiro, demonstrando que o vocabulário financeiro contém terminologia técnica não adequadamente capturada por modelos treinados em corpora gerais. Embora o foco dos autores seja em modelos de linguagem neural, o resultado reforça um princípio que também fundamenta a *FinScript*: ferramentas genéricas são insuficientes para o domínio financeiro, e a especialização é necessária para atingir a precisão exigida pelo setor.

V. GRAMÁTICA FORMAL DA FINSCRIPT

Esta seção apresenta o rascunho inicial da gramática formal da *FinScript*, descrita primeiramente em BNF (*Backus-Naur Form*) e, em seguida, estendida para EBNF (*Extended Backus-Naur Form*). A gramática cobre os construtores centrais da linguagem: declaração de ativos, composição de portfólios, parâmetros de simulação e geração de relatórios.

A. Fundamentos: BNF e EBNF

BNF é uma notação formal para descrever gramáticas livres de contexto, introduzida por John Backus e Peter Naur na especificação da linguagem Algol 60. Cada regra de produção tem a forma $\langle \text{símbolo} \rangle ::= \text{definição}$, onde o lado esquerdo é um não-terminal e o lado direito é uma sequência de terminais e não-terminais; o operador $|$ indica alternativas.

EBNF estende BNF com três operadores adicionais que eliminam a necessidade de recursão explícita para expressar repetição e opcionalidade: colchetes $[]$ indicam zero ou uma ocorrência; chaves $\{ \}$ indicam zero ou mais repetições; e parênteses $()$ agrupam alternativas. A Tabela I resume a notação utilizada neste artigo.

B. Gramática em BNF

A seguir apresenta-se a gramática da *FinScript* em BNF pura, sem os operadores de extensão. Todas as regras que expressam opcionalidade ou repetição são escritas via recursão explícita.

$\langle \text{programa} \rangle$	$::= \langle \text{declaracao} \rangle \mid \langle \text{programa} \rangle \langle \text{declaracao} \rangle$
$\langle \text{declaracao} \rangle$	$::= \langle \text{decl-ativo} \rangle$

TABLE I
NOTAÇÃO EBNF ADOTADA

Símbolo	Significado
$::=$	Define a produção
$ $	Alternativa (ou)
$[]$	Opcional (0 ou 1 vez)
$\{ \}$	Repetição (0 ou mais vezes)
$()$	Agrupamento
$" "$	Terminal literal

	$ \langle \text{decl-portfolio} \rangle$
	$ \langle \text{cmd-simular} \rangle$
	$ \langle \text{cmd-relatorio} \rangle$
	$ \langle \text{cmd-comparar} \rangle$
$\langle \text{decl-ativo} \rangle$	$::= "ativo" \langle id \rangle "{"$ $ "tipo" ":" <$ $ \langle \text{tipo-ativo} \rangle$ $ "quantidade" ":" <$ $ numero >$ $ "preco_medio" ":" <$ $ numero >$ $ \langle \text{campos-opt} \rangle$ $ "}"$
$\langle \text{campos-opt} \rangle$	$::= \mid \langle \text{campos-opt} \rangle \langle \text{campo-}$ $ extra \rangle$
$\langle \text{campo-extra} \rangle$	$::= "taxa" ":" <numero >$ $ "%"$ $ \mid "vencimento" ":" <data >$ $ \mid "indexador" ":" <$ $ indexador >$
$\langle \text{tipo-ativo} \rangle$	$::= "renda_fixa" \mid "acao" \mid "$ $ fundo"$
$\langle \text{indexador} \rangle$	$::= "CDI" \mid "IPCA" \mid "SELIC"$ $ \mid "prefixado"$
$\langle \text{decl-portfolio} \rangle$	$::= "portfolio" \langle id \rangle "{"$ $ \langle \text{lista-ativos} \rangle$ $ "}"$
$\langle \text{lista-ativos} \rangle$	$::= \langle \text{ref-ativo} \rangle \mid \langle \text{lista-}$ $ ativos \rangle \langle \text{ref-ativo} \rangle$
$\langle \text{ref-ativo} \rangle$	$::= "ativo" ":" \langle id \rangle$
$\langle \text{cmd-simular} \rangle$	$::= "simular" \langle id \rangle$ $ "periodo" ":" <numero >$ $ "> "meses"$
$\langle \text{cmd-relatorio} \rangle$	$::= "relatorio" \langle id \rangle "$ $ rentabilidade"$
$\langle \text{cmd-comparar} \rangle$	$::= "comparar" \langle id \rangle "com" \langle id$ $ ">$
$\langle id \rangle$	$::= \langle \text{letra} \rangle \mid \langle id \rangle \langle \text{letra-ou-}$ $ digito \rangle$
$\langle \text{numero} \rangle$	$::= \langle \text{digitos} \rangle \mid \langle \text{digitos} \rangle "."$ $ \langle \text{digitos} \rangle$

```

<digitos>      ::= <digito> | <digitos> <
  digito>

<data>         ::= <digitos> "/" <digitos> "
  "/" <digitos>

<letra>        ::= "a" | ... | "z" | "A" |
  ... | "Z" | "_"

<digito>       ::= "0" | "1" | ... | "9"

```

Listing 1. Gramática da FinScript em BNF

C. Gramática em EBNF

A versão EBNF a seguir é semanticamente equivalente à BNF apresentada acima, porém mais compacta: recursões explícitas para listas e elementos opcionais são substituídas pelos operadores `{ }` e `[ ]`, aproximando a notação da prática comum em ferramentas de geração de *parsers*.

```

programa      ::= { declaracao }

declaracao    ::= decl-ativo
  | decl-portfolio
  | cmd-simular
  | cmd-relatorio
  | cmd-comparar

decl-ativo    ::= "ativo" id "{"
  "tipo"      ":" tipo-ativo
  "quantidade" ":"
    numero
  "preco_medio" ":"
    numero
  [ "taxa"      ":"
    numero "%" ]
  [ "vencimento" ":" data
  ]
  [ "indexador" ":"
    indexador ]
  "}"

tipo-ativo    ::= "renda_fixa" | "acao" | "
  fundo"

indexador     ::= "CDI" | "IPCA" | "SELIC" |
  "prefixado"

decl-portfolio ::= "portfolio" id "{"
  { "ativo" ":" id }
  "}"

cmd-simular   ::= "simular" id
  "periodo" ":" numero "
  meses"

cmd-relatorio ::= "relatorio" id "
  rentabilidade"

cmd-comparar  ::= "comparar" id "com" id

id            ::= letra { letra | digito }

numero       ::= digitos [ "." digitos ]

```

```

digitos      ::= digito { digito }

data         ::= digitos "/" digitos "/"

letra        ::= "a" | ... | "z" | "A" | ...
  | "Z" | "_"

digito       ::= "0" | "1" | ... | "9"

```

Listing 2. Gramática da FinScript em EBNF

D. Exemplo de Programa FinScript

O trecho a seguir ilustra um programa válido segundo a gramática definida, declarando dois ativos de renda fixa, compondo um portfólio e solicitando simulação e relatório de rentabilidade.

```

ativo cdb_banco_x {
  tipo      : renda_fixa
  quantidade : 1
  preco_medio : 10000.00
  taxa      : 110 %
  indexador  : CDI
  vencimento : 31/12/2026
}

ativo tesouro_selic {
  tipo      : renda_fixa
  quantidade : 5
  preco_medio : 1243.50
  indexador  : SELIC
  vencimento : 01/03/2029
}

portfolio carteira_conservadora {
  ativo : cdb_banco_x
  ativo : tesouro_selic
}

simular carteira_conservadora
  periodo : 24 meses

relatorio carteira_conservadora rentabilidade

```

Listing 3. Exemplo de programa FinScript válido

E. Discussão da Gramática

A gramática apresentada é intencionalmente minimalista nesta fase inicial, cobrindo o núcleo expressivo da *FinScript*. Algumas decisões de projeto merecem destaque.

Primeiro, os campos opcionais de um ativo (taxa, vencimento e indexador) foram modelados com o operador `[ ]` da EBNF, pois sua presença depende do tipo do ativo: ativos de renda fixa tipicamente requerem taxa e indexador, enquanto ações podem dispensar ambos. A verificação dessa dependência é semântica, não sintática, e será tratada na fase de análise semântica com regras de negócio do domínio.

Segundo, o não-terminal `decl-portfolio` utiliza repetição `{ "ativo" ":" id }` para permitir portfólios

com número arbitrário de ativos, refletindo a realidade do mercado financeiro onde uma carteira pode conter dezenas de posições.

Terceiro, a gramática é livre de contexto (GLC) e, portanto, analisável por um *parser* descendente recursivo sem retrospecção, desde que as primeiras palavras de cada alternativa sejam disjuntas — condição satisfeita pelos terminais iniciais de cada regra de declaração ("ativo", "portfolio", "simular", "relatorio", "comparar").

Nas próximas fases do projeto, a gramática será estendida com suporte a expressões aritméticas para cálculo de rentabilidade composta, estruturas condicionais (*se/entao*) para alertas semânticos e a instrução *definir* para parametrização de simulações.

VI. IMPLEMENTAÇÃO DO INTERPRETADOR

Esta seção descreve a implementação do interpretador da *FinScript*, detalhando as fases de análise léxica, sintática, semântica e execução, bem como a arquitetura de integração entre o motor Python e a interface web.

A. Análise Léxica

O analisador léxico (*lexer*) percorre o código-fonte caractere a caractere, agrupando-os em *tokens* tipados. Foram definidos 23 tipos de *tokens*, incluindo palavras reservadas (KEYWORD), identificadores (ID), literais numéricos (NUMBER), datas (DATE) e operadores aritméticos e relacionais.

A tabela de palavras reservadas (*keywords*) contempla os construtores da linguagem (*ativo*, *portfolio*, *simular*, *relatorio*, *comparar*), os tipos de ativos (*renda_fixa*, *acao*, *fundo*), os indexadores financeiros (CDI, IPCA, SELIC, prefixado) e as construções de controle (*definir*, *se*, *entao*, *fim*). Identificadores que não pertencem a esse conjunto são classificados como ID, permitindo que o usuário nomeie ativos e portfólios livremente.

O tratamento de datas é realizado dentro do próprio *lexer*: ao encontrar uma sequência de dígitos seguida de /, o analisador consome o restante da data e valida o formato dd/mm/aaaa por expressão regular, lançando *ErroLexico* para formatos inválidos. Comentários iniciados com # são descartados durante a varredura.

B. Análise Sintática

O analisador sintático (*parser*) implementa um *parser* descendente recursivo, onde cada não-terminal da gramática EBNF corresponde diretamente a um método da classe *Parser*. A escolha por *parser* descendente recursivo é motivada pela estrutura LL(1) da gramática da *FinScript*: o primeiro *token* de cada produção é suficiente para determinar unicamente qual regra aplicar, sem necessidade de retrocesso [6].

A construção da AST ocorre simultaneamente à análise sintática. Cada método de reconhecimento instancia o nó correspondente da hierarquia definida em *ast_nos.py*: *NoAtivo*, *NoPortfolio*, *NoSimular*,

NoRelatorio, *NoComparar*, *NoDefinir* e *NoSe* para os construtores principais; *NoExprBinaria*, *NoExprUnaria*, *NoExprNumero* e *NoExprVar* para a hierarquia de expressões aritméticas. A precedência de operadores é implementada pela decomposição clássica em *_expr* (adição/subtração), *_termo* (multiplicação/divisão) e *_fator* (valores atômicos e parênteses).

C. Análise Semântica

A análise semântica opera sobre a AST produzida pelo *parser* e é responsável por verificar restrições que a gramática livre de contexto não é capaz de expressar. A análise é estruturada em três camadas: tabela de símbolos, verificação de tipos e regras de negócio do domínio financeiro. Essa fase é descrita em detalhes na Seção VII.

D. Motor de Execução Financeira

O interpretador percorre a AST em ordem de declaração, executando cada nó conforme sua categoria. Para simulações, o motor aplica a fórmula de juros compostos com capitalização mensal:

$$VF_n = VF_{n-1} \cdot (1 + i_m) + A \quad (1)$$

onde VF_n é o valor do portfólio ao fim do mês n , $i_m = (1 + i_a)^{1/12} - 1$ é a taxa mensal equivalente à taxa anual i_a , e A é o aporte mensal. Para o mês inicial ($n = 0$), tem-se $VF_0 = \text{quantidade} \times \text{preco_medio}$.

A taxa anual efetiva i_a é determinada pelo tipo e indexador do ativo conforme a Tabela II.

TABLE II
DETERMINAÇÃO DA TAXA ANUAL EFETIVA POR TIPO DE ATIVO

Tipo	Indexador	Taxa efetiva
renda_fixa	CDI	$(t/100) \times 10,65\%$
renda_fixa	SELIC	$(t/100) \times 10,75\%$
renda_fixa	IPCA	$4,83\% + t$
renda_fixa	prefixado	t
acao	—	12,00%
fundo	— ($t > 30$)	$(t/100) \times 10,65\%$
fundo	— ($t \leq 30$)	t

Para ativos de *renda_fixa*, o lucro bruto ao final do período é tributado pelo Imposto de Renda conforme a tabela regressiva vigente na legislação brasileira [7]:

$$IR = \begin{cases} 22,5\% \cdot L & \text{se } n \leq 6 \text{ meses} \\ 20,0\% \cdot L & \text{se } 7 \leq n \leq 12 \\ 17,5\% \cdot L & \text{se } 13 \leq n \leq 24 \\ 15,0\% \cdot L & \text{se } n > 24 \text{ meses} \end{cases} \quad (2)$$

onde $L = VF_n - \text{Total Investido}$ é o lucro bruto e n é o prazo em meses. O valor líquido final é $VF_{liq} = VF_n - IR$.

E. Arquitetura de Integração

O sistema adota uma arquitetura cliente-servidor desacoplada. O *backend* consiste em um servidor Flask que expõe o *endpoint* POST `/compilar`, recebendo o código-fonte em JSON, executando as quatro fases do compilador em sequência e retornando a saída textual e os dados de gráficos em JSON. O *frontend* é uma aplicação de página única (SPA) em HTML/JavaScript que permite ao usuário compor programas *FinScript* de forma guiada, envia o código ao *backend* via *fetch* assíncrono e renderiza os resultados com Chart.js. A separação entre o motor de compilação e a interface de usuário garante que o interpretador possa ser consumido igualmente por scripts de linha de comando ou outros clientes.

VII. ANÁLISE SEMÂNTICA DA FINSCRIPT

Esta seção descreve a fase de análise semântica do interpretador da *FinScript*, que opera sobre a AST produzida pelo *parser* e é responsável por verificar restrições que a gramática livre de contexto não é capaz de expressar. A análise é estruturada em três camadas complementares: tabela de símbolos, verificação de tipos e regras de negócio do domínio financeiro.

A. Tabela de Símbolos

A tabela de símbolos é a estrutura central da análise semântica. Toda declaração de *ativo* ou *portfolio* gera uma entrada na tabela, registrando o identificador, a categoria do construtor (*ativo* ou *portfolio*) e os atributos associados. Ao encontrar um identificador em comandos como *simular*, *relatório* ou *comparar*, o analisador consulta a tabela e emite erro semântico caso o símbolo não tenha sido previamente declarado.

Da mesma forma, cada referência *ativo* : `<id>` dentro de um bloco *portfolio* é verificada contra a tabela: referenciar um ativo inexistente configura uso de símbolo não declarado, erro que a análise sintática não detecta, pois a gramática apenas exige que `<id>` seja um identificador bem-formado.

B. Verificação de Tipos e Campos Obrigatórios por Tipo de Ativo

A gramática EBNF modela os campos *taxa*, *vencimento* e *indexador* como opcionais com o operador `[ ]`, pois sua obrigatoriedade depende do valor de *tipo* — uma dependência contextual que caracteriza erro semântico, não sintático. As regras verificadas pelo analisador são:

- **renda_fixa**: exige obrigatoriamente *taxa*, *indexador* e *vencimento*. A ausência de qualquer um desses campos para um ativo desse tipo constitui erro semântico;
- **acao**: os campos *taxa* e *indexador* não se aplicam ao modelo de renda variável; sua presença gera alerta semântico de campo incompatível com o tipo declarado;

- **fundo**: aceita *taxa* e *vencimento* condicionalmente, dependendo da classe do fundo; o analisador emite alerta quando *indexador* é informado sem *taxa* associada.

Adicionalmente, o campo *indexador* é validado quanto à compatibilidade com o tipo do ativo: os valores CDI, IPCA e SELIC são semanticamente válidos apenas para ativos do tipo *renda_fixa*. Atribuir *indexador* : CDI a um ativo do tipo *acao* constitui erro de domínio detectado nesta fase.

C. Validação de Valores Numéricos

Os campos numéricos da linguagem estão sujeitos a restrições de domínio verificadas semanticamente:

- **quantidade**: deve ser um inteiro positivo; valores zero ou negativos são rejeitados;
- **preco_medio**: deve ser estritamente positivo;
- **taxa**: deve situar-se em um intervalo razoável para o mercado financeiro brasileiro — valores negativos ou excessivamente altos (e.g., acima de 500% do CDI) geram alerta semântico;
- **periodo** (em *cmd-simular*): deve ser um inteiro positivo, pois períodos de simulação nulos ou negativos são semanticamente inválidos.

D. Validação de Datas

O não-terminal *data* da gramática aceita qualquer sequência no formato `dd/mm/aaaa`, mas a correção calendarária é verificada semanticamente. O analisador rejeita datas inexistentes, como `31/02/2026`, e emite alerta quando a data de *vencimento* é anterior ou igual à data corrente da simulação, indicando que o ativo já se encontra vencido.

E. Validação de Comandos por Categoria de Símbolo

Os comandos da *FinScript* operam sobre categorias específicas de símbolos. O analisador semântico verifica:

- **simular** e **relatorio**: o identificador fornecido deve referenciar um *portfolio*, não um *ativo* isolado; aplicar esses comandos a um ativo individual constitui erro de categoria;
- **comparar**: ambos os identificadores devem referenciar portfólios; comparar um portfólio com um ativo avulso é erro semântico de categoria.

F. Regras de Negócio: Concentração de Risco

Além das verificações estruturais, o analisador implementa uma regra de negócio relevante ao domínio financeiro: a detecção de concentração de risco em portfólios. Para cada portfólio, o analisador calcula o valor financeiro de cada ativo componente — $\text{quantidade} \times \text{preco_medio}$ — e verifica se algum ativo representa uma participação acima de um limiar configurável (por exemplo, 80% do valor total da carteira). Quando esse limiar é excedido, o analisador emite um alerta semântico informando o identificador do ativo e o percentual de concentração calculado. Essa verificação não impede a execução do programa, mas sinaliza ao usuário um risco de concentração não recomendado pelas boas práticas de gestão de carteiras.

G. Síntese das Camadas Semânticas

A Tabela III sintetiza as três camadas de análise semântica implementadas na *FinScript*, relacionando cada camada às verificações que realiza.

TABLE III
CAMADAS DA ANÁLISE SEMÂNTICA DA FINSCRIPT

Camada	Verificações
Tabela de símbolos	Declaração e referência de identificadores; uso de símbolos não declarados
Verificação de tipos	Campos obrigatórios por tipo de ativo; indexador compatível com o tipo; categoria correta nos comandos
Regras de negócio	Faixas válidas de valores numéricos; validade calendária de datas; concentração de risco em portfólios

VIII. RESULTADOS E CASOS DE USO

Esta seção apresenta os resultados obtidos com a execução de casos de uso representativos do mercado financeiro brasileiro, validando as funcionalidades implementadas no interpretador da *FinScript*.

A. Caso de Uso 1: Portfólio Conservador com Renda Fixa

O primeiro caso de uso declara um portfólio composto por dois ativos de renda fixa — um CDB indexado ao CDI e uma posição no Tesouro SELIC — e solicita simulação por 24 meses com aporte mensal de R\$ 500,00.

```

1 ativo cdb_banco_x {
2   tipo      : renda_fixa
3   quantidade : 1
4   preco_medio : 10000.00
5   taxa      : 110 %
6   indexador  : CDI
7   vencimento : 31/12/2027
8   aporte_mensal : 500
9 }
10
11 ativo tesouro_selic {
12   tipo      : renda_fixa
13   quantidade : 5
14   preco_medio : 1243.50
15   taxa      : 100 %
16   indexador  : SELIC
17   vencimento : 01/03/2029
18   aporte_mensal : 0
19 }
20
21 portfolio carteira_conservadora {
22   ativo : cdb_banco_x
23   ativo : tesouro_selic
24 }
25
26 simular carteira_conservadora periodo : 24
27   meses
28 relatorio carteira_conservadora rentabilidade

```

Listing 4. Caso de uso 1: portfólio conservador

Após 24 meses, o CDB (110% do CDI) apresentou rentabilidade líquida de aproximadamente 21,8% sobre o total investido, já descontado o IR de 17,5% incidente sobre o lucro. O Tesouro SELIC, com taxa a 100% da SELIC, atingiu rentabilidade líquida de cerca de 18,9% no mesmo período, com IR de 17,5%. O interpretador gerou corretamente o histórico mensal para renderização de gráfico de linha, além do relatório tabular de composição de carteira.

B. Caso de Uso 2: Portfólio Arrojado com Renda Variável

O segundo caso de uso compõe um portfólio misto de ações e fundo multimercado, demonstrando o tratamento de diferentes classes de ativos e a comparação entre dois portfólios.

```

1 ativo acoes_vale {
2   tipo      : acao
3   quantidade : 50
4   preco_medio : 70.00
5   aporte_mensal : 100
6 }
7
8 ativo fundo_multimercado {
9   tipo      : fundo
10  quantidade : 100
11  preco_medio : 50.00
12  taxa      : 110 %
13  aporte_mensal : 200
14 }
15
16 portfolio carteira_arrojada {
17   ativo : acoes_vale
18   ativo : fundo_multimercado
19 }
20
21 comparar carteira_conservadora com
22   carteira_arrojada

```

Listing 5. Caso de uso 2: portfólio arrojado e comparação

O comando `comparar` projetou os dois portfólios por 12 meses e emitiu o veredito: a carteira_arrojada superou a carteira_conservadora em 1,29 pontos percentuais no período, resultado esperado dado o horizonte curto e a ausência de IR sobre renda variável. O gráfico de pizza gerado ilustrou a distribuição de patrimônio final entre os dois portfólios.

C. Caso de Uso 3: Uso de Condicionais e Variáveis

O terceiro caso de uso demonstra as construções `definir` e `se/entao/fim`, permitindo que a simulação seja condicionada a parâmetros configuráveis.

```

1 definir meta_taxa = 10.5
2 definir horizonte = 12 + 12
3
4 se meta_taxa >= 10.5 entao
5   simular carteira_conservadora
6     periodo : horizonte meses
7 fim

```

Listing 6. Caso de uso 3: condicional e variáveis

Quando a variável `meta_taxa` satisfaz a condição (`>= 10.5`), o bloco `se` é avaliado como verdadeiro durante a

análise semântica e o comando `simular` é executado pelo interpretador. Ao modificar `meta_taxa` para um valor inferior, o bloco é ignorado completamente — nenhum símbolo interno é registrado na tabela de símbolos nem executado, comportamento verificado por teste unitário. A expressão aritmética `12 + 12` é avaliada para 24 durante a análise semântica, antes da execução.

D. Validação Semântica: Detecção de Erros

A robustez do analisador semântico foi verificada com entradas intencionalmente inválidas. A Tabela IV sumariza os principais erros detectados e as mensagens geradas.

TABLE IV
EXEMPLOS DE ERROS SEMÂNTICOS DETECTADOS PELA FINSCRIPT

Entrada inválida	Erro emitido
<code>acao com indexador: CDI</code>	Indexador CDI é exclusivo de <code>renda_fixa</code>
<code>renda_fixa sem vencimento</code>	Campo obrigatório <code>vencimento</code> ausente
<code>simular referenciando um ativo</code>	Identificador não é um portfólio
<code>comparar com portfólio não declarado</code>	Símbolo não declarado na tabela
<code>quantidade: 10.5</code>	Quantidade deve ser inteiro positivo
<code>vencimento: 01/01/2020</code>	Alerta: ativo com vencimento passado
<code>Divisão por zero em definir</code>	Erro semântico: divisão por zero

Todos os erros foram detectados na fase correta — erros de formato em `vencimento` no léxico, referências inválidas no semântico — sem propagar para a fase de execução, o que demonstra a separação adequada de responsabilidades entre as fases do compilador.

IX. COMPARAÇÃO COM TRABALHOS RELACIONADOS

Esta seção posiciona a *FinScript* em relação aos trabalhos mais relevantes da literatura, identificando convergências, diferenças de escopo e contribuições originais.

A. Comparação com a Abordagem de Contratos Financeiros (Peyton Jones et al., 2000)

O trabalho seminal de [2] propõe uma DSL *interna* em Haskell baseada em combinadores para descrição de contratos financeiros. A abordagem é formalmente elegante e permite composição arbitrária de instrumentos, mas pressupõe um usuário com fluência em programação funcional e no sistema de tipos de Haskell. A *FinScript*, por ser uma DSL *externa* com sintaxe declarativa próxima ao vocabulário do domínio, é acessível a analistas financeiros sem formação em programação. Enquanto [2] foca na descrição semântica de contratos (precificação, composição, avaliação estocástica), a *FinScript* foca na simulação de portfólios e geração de relatórios — escopos complementares, não concorrentes.

B. Comparação com a Finance Modeling Language (Cloudflight, 2023)

A *Finance Modeling Language* (FML) descrita em [3] é uma DSL de uso corporativo desenvolvida para modelar produtos financeiros em escala industrial. A FML opera em um contexto de implantação empresarial, com equipes dedicadas, integração a sistemas legados e requisitos de governança que extrapolam o escopo acadêmico. A *FinScript*, por outro lado, é um projeto de prova de conceito autocontido que demonstra a aplicabilidade das técnicas de compiladores ao domínio financeiro brasileiro, incluindo particularidades do mercado local como a tabela regressiva de IR e os indexadores CDI, IPCA e SELIC. A contribuição metodológica da *FinScript* reside na explicitação das fases clássicas de compilação como artefatos auditáveis, o que a torna adequada a contextos educacionais e de pesquisa.

C. Comparação com SWEL (Hernández et al., 2023)

A SWEL [4] é uma DSL para especificação de fluxos de trabalho científicos intensivos em dados, desenvolvida segundo princípios de engenharia orientada a modelos (*Model-Driven Engineering*). Embora o domínio seja distinto, a metodologia de desenvolvimento é análoga à adotada na *FinScript*: identificação dos construtores centrais do domínio, definição de gramática formal e validação por casos de uso reais. Uma diferença relevante é que SWEL adota uma abordagem de metamodelo com geração de código, enquanto a *FinScript* implementa diretamente um interpretador em Python, priorizando a transparência das fases de compilação sobre a portabilidade de plataforma.

D. Comparação com Ferramentas de Análise Financeira em Python

Bibliotecas Python como `pandas` [8] e `numpy` [9] são amplamente utilizadas para análise financeira programática. Embora ofereçam grande flexibilidade, elas exigem que o analista expresse operações financeiras no vocabulário da linguagem hospedeira (Python), misturando lógica de negócio com detalhes de implementação. Um portfólio de renda fixa indexada ao CDI, com aporte mensal e desconto de IR regressivo, requer dezenas de linhas de código Python com tratamento manual de convenções de mercado. Na *FinScript*, o mesmo portfólio é expresso em aproximadamente 10 linhas de código declarativo, com todas as regras de negócio encapsuladas no interpretador. Essa diferença de expressividade é precisamente o benefício que [1] identificam como a principal motivação para o desenvolvimento de DSLs de domínio.

A Tabela V sintetiza a comparação entre a *FinScript* e os trabalhos relacionados nas dimensões mais relevantes.

X. CONCLUSÃO

Este artigo apresentou o projeto e a implementação da *FinScript*, uma Linguagem de Domínio Específico para processamento de dados financeiros. Partindo da definição formal da gramática em BNF e EBNF, o trabalho contemplou as quatro fases clássicas de um compilador — análise léxica, análise

TABLE V
COMPARAÇÃO ENTRE FINSCRIPT E TRABALHOS RELACIONADOS

Cr��rio	Fin-Script	PJ2000	FML	pandas
Tipo de DSL	Externa	Interna	Externa	Interna
Sintaxe declarativa	Sim	N��o	Sim	N��o
Verifica��o sem��ntica	Sim	Parcial	Sim	N��o
Mercado BR (CDI/IR)	Sim	N��o	N��o	Manual
Condicional na DSL	Sim	N��o	Sim	N/A
C��digo aberto	Sim	Sim	N��o	Sim

sint  tica por *parser* descendente recursivo, an  lise sem  ntica com tabela de s  mbolos e verifica  o de regras de neg  cio, e execu  o via caminhamento na AST — resultando em um interpretador funcional capaz de simular portf  lios, gerar relat  rios e comparar estrat  gias de investimento do mercado financeiro brasileiro.

Os casos de uso apresentados demonstraram que a *FinScript*    capaz de expressar, em poucas linhas de c  digo declarativo, opera  es que exigiriam dezenas de linhas em Python gen  rico, validando a premissa de que DSLs de dom  nio oferecem ganhos reais de expressividade e redu  o de erros quando o dom  nio    suficientemente estruturado. As verifica  es sem  nticas implementadas — obrigatoriedade de campos por tipo de ativo, compatibilidade de indexadores, valida  o de datas e detec  o de concentra  o de risco — demonstraram que a an  lise sem  ntica    o locus natural para a codifica  o de regras de neg  cio em compiladores de dom  nio.

A compara  o com trabalhos relacionados evidenciou que a *FinScript* ocupa um espa  o distinto no espectro de ferramentas de an  lise financeira program  tica: ao contr  rio de DSLs baseadas em combinadores [2],    acess  vel sem forma  o em programa  o funcional; ao contr  rio de bibliotecas de prop  sito geral [8], encapsula as conven  es do mercado brasileiro (CDI, SELIC, IPCA, IR regressivo) na pr  pria sem  ntica da linguagem; e ao contr  rio de solu  es industriais [3],    audit  vel e reprodu  vel como artefato acad  mico.

Como trabalhos futuros, identificam-se tr  s dire  es naturais de extens  o. Primeiro, a adi  o de estruturas de controle iterativas (*para / enquanto*) permitiria expressar estrat  gias de rebalanceamento per  dico de portf  lios diretamente na linguagem. Segundo, a integra  o com APIs de dados de mercado em tempo real (e.g., o *endpoint* do Banco Central para CDI e IPCA) eliminaria a depend  ncia de taxas est  ticas no interpretador, aumentando o realismo das simula  es. Terceiro, a gera  o de c  digo Python a partir da AST — em substitui  o    interpreta  o direta — abriria caminho para a integra  o com ecossistemas de an  lise de dados existentes, como *pandas* e *matplotlib*, ampliando o alcance da linguagem para contextos profissionais.

REFERENCES

- [1] M. Mernik, J. Heering, and A. M. Sloane, “When and how to develop domain-specific languages,” *ACM Computing Surveys*, vol. 37, no. 4, pp. 316–344, 2005.
- [2] S. L. Peyton Jones, J.-M. Eber, and J. Seward, “Composing contracts: an adventure in financial engineering,” in *Proceedings of the ACM SIGPLAN International Conference on Functional Programming (ICFP’00)*. New York: ACM Press, 2000, pp. 280–292.
- [3] Cloudflight, “Optimizing financial services with Domain Specific Language (DSL),” Case Study. Dispon  vel em: <https://www.cloudflight.io/en/project/optimizing-financial-services-with-domain-specific-language-dsl/>, 2023, acesso em: abr. 2026.
- [4] J. M. Hern  ndez, J. Garc  a-Garc  a, and A. Ruiz-Gonz  lez, “SWEL: A domain-specific language for modeling data-intensive workflows,” *Business & Information Systems Engineering*, vol. 65, pp. 649–672, 2023.
- [5] H. Sakaji, H. Sakai, and K. Izumi, “Constructing and analyzing domain-specific language model for financial text mining,” *Information Processing & Management*, vol. 60, no. 2, 2023.
- [6] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston: Pearson Addison-Wesley, 2006.
- [7] Receita Federal do Brasil, “Tributa  o de aplica  es financeiras de renda fixa,” Dispon  vel em: <https://www.gov.br/receitafederal/pt-br>, 2024, acesso em: abr. 2026.
- [8] The pandas development team, “pandas-dev/pandas: Pandas,” in *Proceedings of the 9th Python in Science Conference*, S. van der Walt and J. Millman, Eds., 2020, pp. 56–61.
- [9] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.